

Ambiance des lieux — Phase 3

IFT3225 — Technologies internet
Université de Montréal

Équipe

Aguibou FOFANA — 20332292
Mamadou TRAORE — 20290120
Kofi OSEL — 20272184

4 août 2026

Table des matières

1. Fonctionnalité additionnelle.....	3
1.1 « Où aller ? » — recommandation en direct.....	3
1.2 Justification.....	3
1.3 Effet « prise en direct » sur la carte.....	3
2. Maintenabilité et réutilisabilité.....	3
2.1 Séparation des responsabilités au backend.....	3
2.2 Contexte, hooks et store au frontend.....	4
3. Tests unitaires des services.....	4
4. Stratégie de cache.....	5
4.1 Côté backend (serveur Express).....	5
4.2 Côté frontend (client React).....	5
4.3 Ce qui n'est jamais mis en cache.....	5
5. Déploiement.....	6
6. Optimisations et faiblesses restantes.....	6
6.1 Optimisations mises en place.....	6
6.2 Faiblesses restantes.....	6
7. Utilisabilité et vidéo.....	7

1. Fonctionnalité additionnelle

La Phase 3 ajoute une fonctionnalité qui améliore concrètement l'expérience utilisateur tout en restant dans le domaine de l'application : la recommandation « Où aller ? » et un retour visuel de collecte en direct sur la carte.

1.1 « Où aller ? » — recommandation en direct

Un bouton « Où aller ? » (accessible connecté ou non) ouvre une vue qui classe tous les lieux du plus calme au plus animé, en direct, met en avant le lieu recommandé et permet de choisir la fenêtre d'analyse (15 min / 30 min / 1 h). La liste se rafraîchit automatiquement via le **flux temps réel (SSE)**. Elle s'appuie sur un nouvel endpoint **GET /v1/ambiance/where-to-go**, filtrable par ville et par type.

1.2 Justification

Le besoin réel de l'utilisateur n'est pas « quelle est l'ambiance du lieu X ? » mais « **où puis-je aller maintenant ?** ». Auparavant, il fallait ouvrir chaque lieu un par un pour comparer. La fonctionnalité répond directement à cette décision et réutilise l'infrastructure existante : la fonction pure **rankByAmbiance** (partagée avec /compare et couverte par les tests) et le bus d'événements SSE. Le gain d'expérience est important pour très peu de code neuf, et la logique de classement, isolée et testée, évite toute duplication entre la route de comparaison et la nouvelle fonctionnalité.

1.3 Effet « prise en direct » sur la carte

Pendant une collecte, dès qu'une mesure ou une observation arrive en temps réel, le lieu concerné réagit visuellement sur la carte pendant quelques secondes : anneau qui se propage autour du marqueur (couleur de l'ambiance), légère vibration du point, badge « Prise en direct » listant le(s) lieu(x) en captation, et retour haptique (**navigator.vibrate**) sur mobile compatible. L'effet s'éteint automatiquement après quatre secondes et respecte la préférence système « **prefers-reduced-motion** ». Il matérialise concrètement le flux temps réel et rend une session de collecte lisible d'un coup d'œil.

2. Maintenabilité et réutilisabilité

Le découpage sépare l'affichage, la logique et l'accès aux données, et met à profit les outils que React offre pour partager l'état et réutiliser le code. L'objectif est la clarté des frontières, pas le nombre de fichiers.

2.1 Séparation des responsabilités au backend

La logique métier est isolée dans une couche de services (src/services/) composée de fonctions pures, découplées de Mongoose et d'Express. Les routes se limitent à valider les entrées, appeler le service et formater la réponse. Cette séparation rend la logique testable sans base de données ni serveur (voir section 3) et permet sa réutilisation : **rankByAmbiance sert à la fois /compare et /where-to-go**.

2.2 Contexte, hooks et store au frontend

Le composant **App**, auparavant un composant « dieu » d'environ 275 lignes gérant l'authentification, les favoris et la navigation, a été ramené à un orchestrateur d'environ 135 lignes. Chaque outil React est employé là où sa frontière est la plus juste :

Outil	Module	Responsabilité
Contexte	AuthContext	Identité partagée (user, token, login, logout), persistance localStorage, déconnexion automatique sur token expiré.
Contexte	FavoritesContext	Favoris partagés (liste, isFavorite, toggleFavorite), dépend de l'authentification.
Hook	useLocations	Récupération des lieux avec états chargement / erreur / rechargement isolés.
Store (Zustand)	uiStore	État de navigation (vue courante, lieu sélectionné, filtre favoris, écran d'auth) et bannière de notification, gérés hors des composants.
Composants	StateMessage, utils/ambiance	Composants et utilitaires réutilisables (états chargement/erreur, couleurs et libellés d'ambiance) partagés par la carte, le détail et la recommandation.

Le **prop-drilling** a été supprimé : **LocationDetail** et **MyLocations** lisent directement les contextes au lieu de recevoir user, token et favoris en cascade. **MapView** consomme désormais l'utilitaire de couleur partagé, éliminant une table de couleurs dupliquée.

3. Tests unitaires des services

Les services du backend sont couverts par des tests unitaires écrits avec **Vitest** (outil léger et proche de Vite, ajouté au **package.json**). Comme la logique métier est isolée en fonctions pures, les tests s'exécutent sans serveur ni base de données. Ils se lancent par **npm test** et passent tous (33 tests) sur le code livré et déployé.

Service / module	Cas couverts (≥ 3 chacun)
ambianceService	Étiquette d'ambiance selon les seuils, portrait sur fenêtre vide (inconnu), moyenne du bruit et mode des observations, créneaux calmes (seuil, filtre par jour local Montréal), agrégation de l'historique par tranche, classement « où aller » (tri, lieux sans mesure écartés, cas vide).
cacheService	set/get, clé absente, suppression ciblée (del), invalidation par motif (delPattern), expiration au TTL, vidage complet (flushAll).
Middleware de cache	En-tête Cache-Control posé, MISS puis HIT servi depuis le cache, non-mise en cache des réponses non-200, exclusion des écritures, noCache.

Service / module	Cas couverts (≥ 3 chacun)
Utilitaires de temps	parseDuration (unités valides / invalides), isValidDate, buildTimeWindow (last, from/to, combinaison interdite, date invalide).

4. Stratégie de cache

Le cache est mis en œuvre des deux côtés, avec des rôles complémentaires : le frontend évite les allers-retours réseau, le backend évite de recalculer les agrégations MongoDB. Un en-tête HTTP Cache-Control relie les deux.

4.1 Côté backend (serveur Express)

Aspect	Détail
Quoi	Le corps JSON des lectures publiques (GET) : vues d'ambiance, listes measurements, observations, locations, devices.
Où	Cache en mémoire du processus (node-cache), activé par un middleware. Clé = URL complète (req.originalUrl, paramètres inclus).
Combien de temps	Ambiance temps réel (now, compare, where-to-go) 30 s ; history / quiet-hours 5 min ; measurements / observations 60 s ; locations 1 h ; devices 30 min.
Invalidation	À chaque écriture, la route purge le motif concerné (delPattern) — un POST /measurements purge « ambiance » et « measurements ». Comme la clé est l'URL, les entrées correspondantes disparaissent immédiatement. Sinon, expiration au TTL.
Observabilité	En-tête X-Cache: HIT MISS ajouté à chaque réponse cacheable.

4.2 Côté frontend (client React)

Aspect	Détail
Quoi	Les réponses des GET publics, interceptées de façon transparente par axios.
Où	localStorage du navigateur (préfixe ambiance_cache_), clé = URL + paramètres ; persiste entre les rechargements de page.
Combien de temps	TTL par endpoint (mêmes ordres de grandeur que le backend : ambiance 30 s, history/quiet-hours 5 min, locations 1 h...), 5 min par défaut.
Invalidation	Après une écriture réussie, purge par motif des entrées liées (une observation soumise purge « ambiance » et « observations »). Sinon, expiration au TTL.

4.3 Ce qui n'est jamais mis en cache

- Toutes les écritures (POST / PUT / DELETE), ni au front ni au back.
- L'authentification (/v1/auth/*) et les données par utilisateur (favoris, « mes lieux ») : middleware noCache côté serveur, exclusion explicite /auth/ côté client.
- Le flux temps réel SSE (/v1/events), non cacheable par nature.

5. Déploiement

L'application est déployée sur Render en deux services décrits par un **Blueprint (render.yaml)** : un **Web Service Node** pour le backend (API Express) et un **Static Site** pour le frontend (build Vite servi par CDN). La base reste hébergée sur MongoDB Atlas.

Service	Build	Démarrage / publication
Backend	npm install && npm run build	node dist/index.js
Frontend	cd client && npm install && npm run build	publication : client/dist

URLs en ligne : backend <https://ambiance-api.onrender.com> (santé /v1/health), frontend <https://ambiance-client.onrender.com>.

Variables définies dans Render — backend : **MONGODB_URI, DB_NAME, ADMIN_API_KEY, JWT_SECRET** (le PORT est fourni automatiquement) ; frontend : **VITE_API_URL** (URL du backend suivie de /v1). **VITE_API_URL** étant une variable de build, toute modification impose un redéploiement du frontend.

6. Optimisations et faiblesses restantes

6.1 Optimisations mises en place

- **Cache à deux niveaux** (section 4) : mémoire serveur et **localStorage** client, reliés par **Cache-Control**, avec invalidation ciblée sur les écritures.
- **Index MongoDB composés** { locationSlug, timestamp } sur measurements et observations : les requêtes par lieu et fenêtre temporelle sont servies par index, sans balayage complet.
- **Code-splitting** (React.lazy + Suspense) : la carte (Leaflet) et le détail (Chart.js) sont chargés à la demande dans des chunks séparés ; le bundle initial passe d'environ 587 kB à environ 251 kB, et Chart.js n'est chargé qu'à l'ouverture d'un lieu.
- **Requêtes parallélisées** (Promise.all), **pagination bornée** et **rate limiting** (130 req/min) pour protéger le serveur.
- **Temps réel par push (SSE)** plutôt que polling : seul le lieu concerné est rafraîchi à chaque nouvelle donnée.
- **Maintenabilité** : logique pure testée et réutilisée ; frontend découpé en contexte, hooks, store et composants réutilisables (section 2).

6.2 Faiblesses restantes

- **Faible volontaire POST /devices** non authentifié : conservée par choix pédagogique ; correctif connu (middleware admin).
- **Cache serveur en mémoire du processus** : perdu au redémarrage et non partagé en cas de scaling horizontal ; piste : Redis.

- **Tension cache / temps réel** : un rafraîchissement déclenché par un événement SSE venant d'un autre client peut servir une donnée périmée jusqu'au TTL (≤ 30 s), l'invalidation frontend ne se déclenchant que sur les écritures locales.
- **Requêtes N+1** dans /where-to-go et /compare : une paire de requêtes par lieu ; acceptable au volume actuel, à revoir si le nombre de lieux croît.
- **Couverture de tests partielle** : services couverts, mais pas les routes (pas de tests d'intégration) ni le frontend.
- **Authentification sans refresh token** (JWT 7 jours) et **CORS ouvert** à toutes les origines : acceptables pour la démo, à restreindre en production.

7. Utilisabilité et vidéo

L'utilisabilité est traitée de façon transverse. Chaque vue distingue explicitement les états de chargement, d'erreur et de succès (composants d'état réutilisables, messages en français issus de l'API, confirmations d'action). La réactivité repose sur les requêtes parallélisées, le cache à deux niveaux, le code-splitting et le flux temps réel qui rafraîchit sans rechargement. L'interface reste claire : navigation par onglets, légende permanente, retours visuels immédiats (effet « prise en direct », indicateur de mise à jour en direct).

Une vidéo de démonstration de 8 à 10 minutes accompagne la remise : elle présente la carte et les états d'ambiance, la recommandation « Où aller ? », le compte utilisateur et les actions protégées, le temps réel et l'effet de collecte en direct, puis un aperçu de l'architecture (tests, cache, déploiement).